NETGUARDAI: AI-DRIVEN NETWORK TRAFFIC ANALYSIS AND INTRUSION DETECTION SYSTEM

Organization: Megaminds IT Services

Project Title: AI-Driven Network Traffic Analysis and Intrusion Detection System

Student Name: Pradnya Pathade

Internship Role: Cybersecurity Intern

Submission Date: 25/10/2026

1. Abstract

Network security is an important part of protecting computer systems, applications and organizational information. Networks generate traffic records that may contain indications of suspicious activity, such as unusually high connection counts, repeated access to different destination ports or unexpected connection patterns. Manually reviewing large numbers of records can be time-consuming.

NetGuardAI is a Python-based network traffic analysis and intrusion detection prototype developed as part of a cybersecurity internship at Megaminds IT Services. The application uses a Streamlit dashboard and pandas to process uploaded CSV traffic records. It applies rule-based detection to identify potentially suspicious patterns and presents the results through status labels, explanations, summary metrics, charts and filtering options. Users can also download the analyzed results as a CSV file.

The prototype was tested using simulated traffic records representing normal activity, scan-like behaviour, high connection counts, selected destination ports, ambiguous cases and missing data. The project demonstrates how basic traffic analysis rules can support preliminary security monitoring. However, the current implementation does not include a functioning machine-learning model or live packet capture. Its results depend on the available data and configured thresholds. Future improvements could include richer traffic features, larger evaluation datasets, live traffic ingestion and machine-learning-based anomaly detection.

2. Introduction

Computer networks are essential for communication, file sharing, web services and access to digital resources. However, network activity may also include suspicious connections, reconnaissance attempts, unusually high traffic volumes and other patterns that require investigation.

Intrusion detection systems help identify activity that may indicate a security problem. They can use predefined rules, known attack patterns, statistical methods or machine-learning techniques to examine network information and generate alerts.

NetGuardAI was developed to demonstrate a simple and understandable approach to network traffic analysis. It accepts traffic records in CSV format, examines selected fields and classifies records according to configured rules. The dashboard helps users review the results without manually inspecting every record.

The current prototype focuses on offline analysis of supplied traffic data rather than continuous monitoring of a live network. It is an educational demonstration of rule-based detection, not a complete enterprise intrusion detection platform.

3. Problem Statement

Network traffic records may contain a mixture of normal and potentially suspicious activity. Reviewing these records manually can be difficult, particularly when many connections are present or when some records contain incomplete information.

A simple analysis tool is therefore useful for identifying records that deserve further investigation. The challenge is to design an accessible application that processes structured traffic data, applies understandable detection rules and presents results in a convenient format.

NetGuardAI addresses this problem by automating basic traffic-record analysis and providing a dashboard for reviewing potentially suspicious activity. It also demonstrates the limitations of threshold-based detection, including false positives and missed detections.

4. Project Objectives

The main objectives of NetGuardAI are:

1. To develop a working network traffic analysis prototype using Python.

2. To accept and process traffic records from CSV files.

3. To examine connection counts, destination ports and distinct destination ports associated with source IP addresses.

4. To identify potentially suspicious records using predefined rules.

5. To display classification results and explanations in a dashboard.

6. To provide summary metrics, charts and status-based filtering.

7. To allow users to download analyzed traffic records.

8. To evaluate the prototype using simulated scenarios and identify possible improvements.

5. Technologies Used

5.1 Python

Python is the primary programming language used to develop the application. It supports data processing, application logic and integration with the dashboard framework.

5.2 Streamlit

Streamlit provides the web-based user interface. It allows users to upload CSV files, view summary metrics, explore charts, filter records and download analysis results.

5.3 Pandas

Pandas is used to load CSV files, process tabular traffic data, convert selected fields into numeric values and prepare the results displayed in the dashboard.

5.4 CSV traffic data

CSV files provide structured input for the prototype. The sample dataset contains fields such as source IP, protocol, destination port, connection count and expected label.

The application depends on the fields available in the uploaded data. It does not automatically obtain information that is absent from the CSV.

6. System Design and Methodology

The prototype follows a straightforward processing workflow.

Step 1: Traffic data input

The user uploads a CSV file through the Streamlit interface. If no file is supplied, the application can use its built-in demonstration records.

Step 2: Data validation and preprocessing

The application checks for required fields, including connection count. Selected fields are converted to numeric values so that rules can be applied consistently. Missing or invalid values may require manual review.

Step 3: Feature examination

The application examines connection count and destination port. It also identifies the number of distinct destination ports associated with a source IP address, where the required information is available.

Step 4: Rule-based detection

The application applies predefined conditions to identify potentially suspicious records. These include unusually high connection counts, selected destination ports and a pattern involving multiple distinct destination ports from one source.

Step 5: Classification and explanation

Records are assigned a status such as "Normal", "Suspicious - Review" or "Needs Review". Explanations describe the applicable rule or data-quality issue.

Step 6: Dashboard presentation

The processed data is displayed in a table alongside summary metrics, charts and filtering options. The user can review individual records and download the analyzed results.

This workflow supports preliminary investigation. A detection status is an indication for review and does not independently establish that an attack has occurred.

7. Detection Rules and Implementation

7.1 High connection count

The current prototype flags a record when its connection count exceeds 100. This threshold is used as a demonstration rule for identifying unusually high counts.

The threshold is not a universal definition of malicious traffic. Legitimate services may generate many connections, while attacks may operate below the threshold.

7.2 Selected destination ports

The prototype flags activity involving destination ports 23, 445 and 3389 for review.

These ports are commonly associated with Telnet, SMB and Remote Desktop Protocol (RDP), respectively. Their use is not inherently malicious. The rule identifies activity for investigation rather than confirming an attack.

7.3 Possible port-scanning pattern

The application examines distinct destination ports associated with each source IP address. A source contacting four or more distinct destination ports is flagged for review under the configured rule.

This may help identify scan-like behaviour. However, legitimate software, administrative tools and ordinary network operations may also contact multiple ports.

7.4 Missing or invalid data

Records with missing or invalid required values may be marked for review. This prevents incomplete data from being treated as confidently normal and highlights the need for data validation.

7.5 Machine-learning status

A machine-learning model was explored during development, but the required library could not be used successfully in the final environment because of a Windows Application Control error. Consequently, the submitted working prototype uses rule-based detection. No functioning machine-learning model or ML-derived accuracy result is claimed in this report.

8. Testing and Demonstration Scenarios

The prototype was tested using simulated CSV records. The scenarios below describe the intended demonstrations; individual outcomes should be reported according to the actual dashboard results.

8.1 Normal traffic

Records with ordinary connection counts and commonly used destination ports are used to demonstrate the normal-traffic baseline. The dashboard displays their assigned statuses so that unexpected alerts can be identified.

8.2 Port-scan-like activity

The sample data includes multiple destination ports associated with the same source IP address. This scenario demonstrates the distinct-port rule and how the system flags possible reconnaissance behaviour.

8.3 Simulated flooding

The sample contains records with high connection counts, including values of 250 and 350. These records are used to demonstrate the high-count rule.

This scenario simulates a possible indicator of flooding. It does not reproduce a real denial-of-service attack or measure packet rates, traffic duration or service availability.

8.4 Additional suspicious-activity scenario

The sample includes activity involving destination port 445. The dashboard can flag this port for review and provide an opportunity to discuss SMB-related security risks.

A port match alone cannot confirm an exploit or a specific attack. Authentication records, protocol details and additional context would be needed to investigate an actual SMB-related incident.

8.5 Ambiguous and incomplete records

A borderline record with a connection count of 35 on destination port 443 demonstrates that traffic context matters. Such activity may be legitimate, and the available fields may be insufficient to establish intent.

The sample also contains a record with a missing connection count. This demonstrates the importance of data validation and manual review.

9. Detection Performance Evaluation

Detection performance should be evaluated by comparing expected labels with actual detector outcomes. The sample dataset contains an expected-label field that can support this comparison when the labels are reliable.

For evaluation, define suspicious records as the positive class and combine the two review statuses as flagged. Records classified as normal are treated as not flagged.

- True positive (TP): A suspicious record is correctly flagged.

- False positive (FP): A normal record is flagged.

- False negative (FN): A suspicious record is classified as normal.

- True negative (TN): A normal record is classified as normal.

Precision measures the proportion of flagged records that are actually labelled suspicious:

Precision = TP / (TP + FP)

Recall measures the proportion of suspicious records that are flagged:

Recall = TP / (TP + FN)

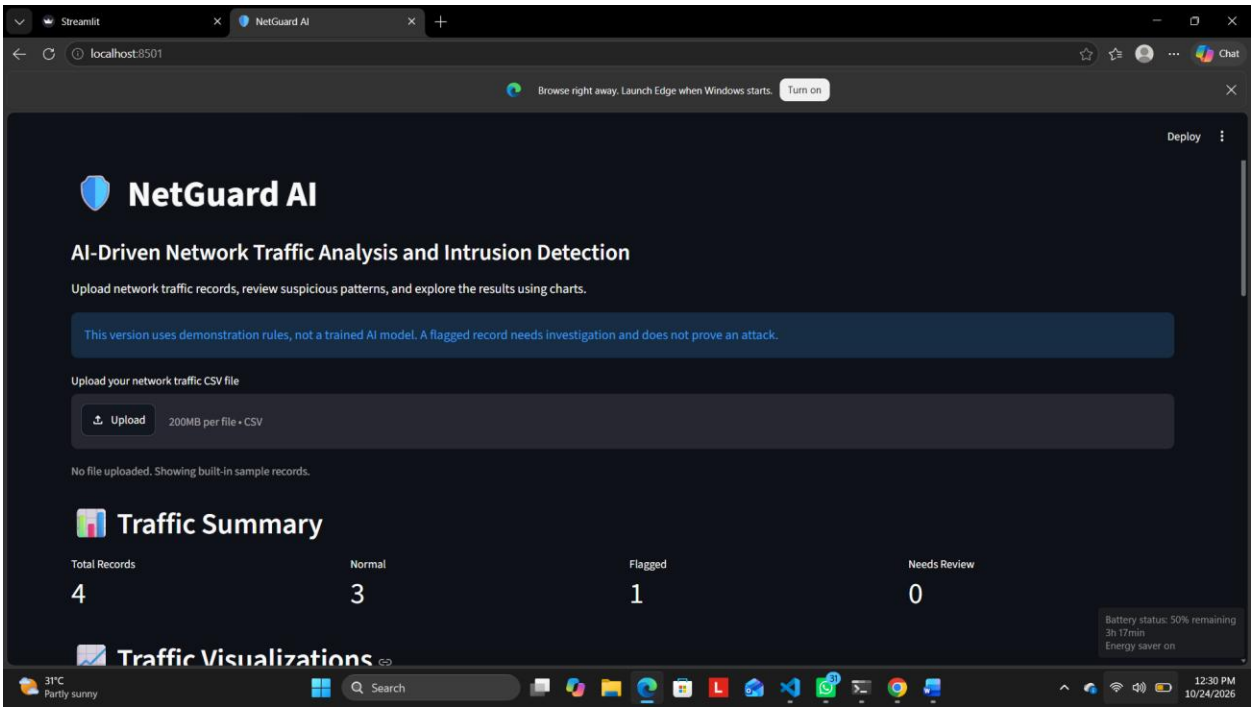
Accuracy measures the proportion of correctly classified records:

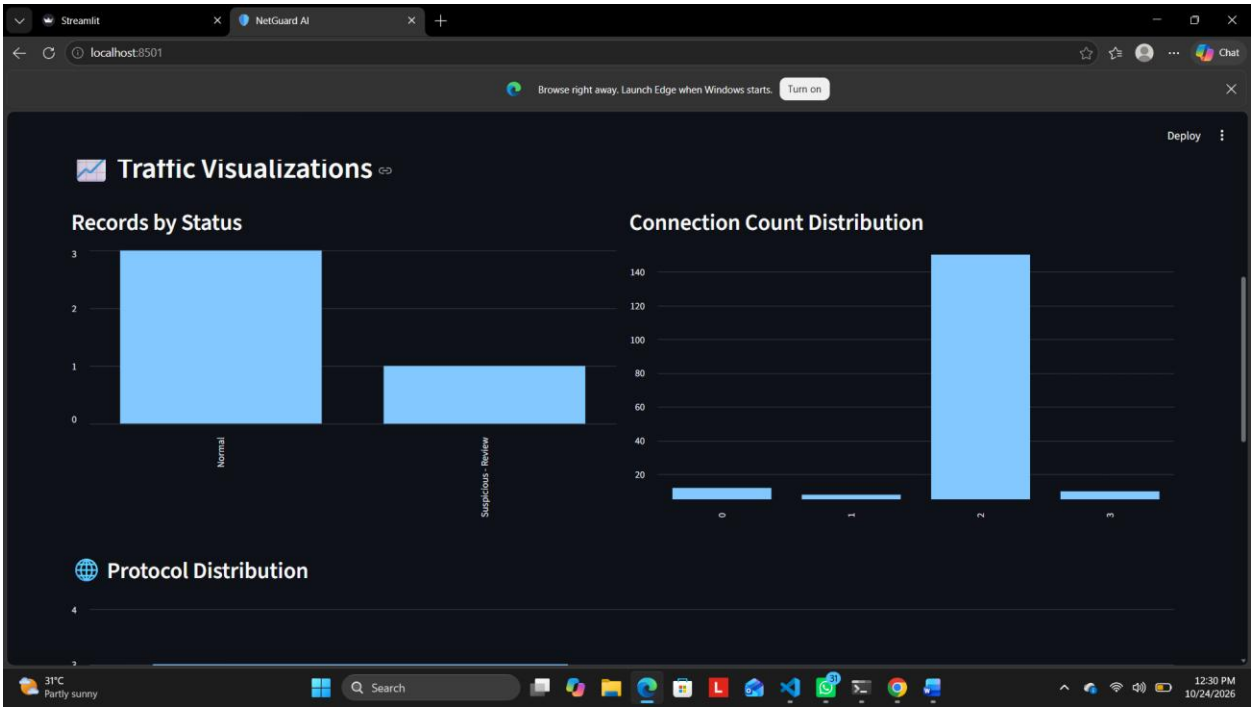
Accuracy = (TP + TN) / (TP + TN + FP + FN)

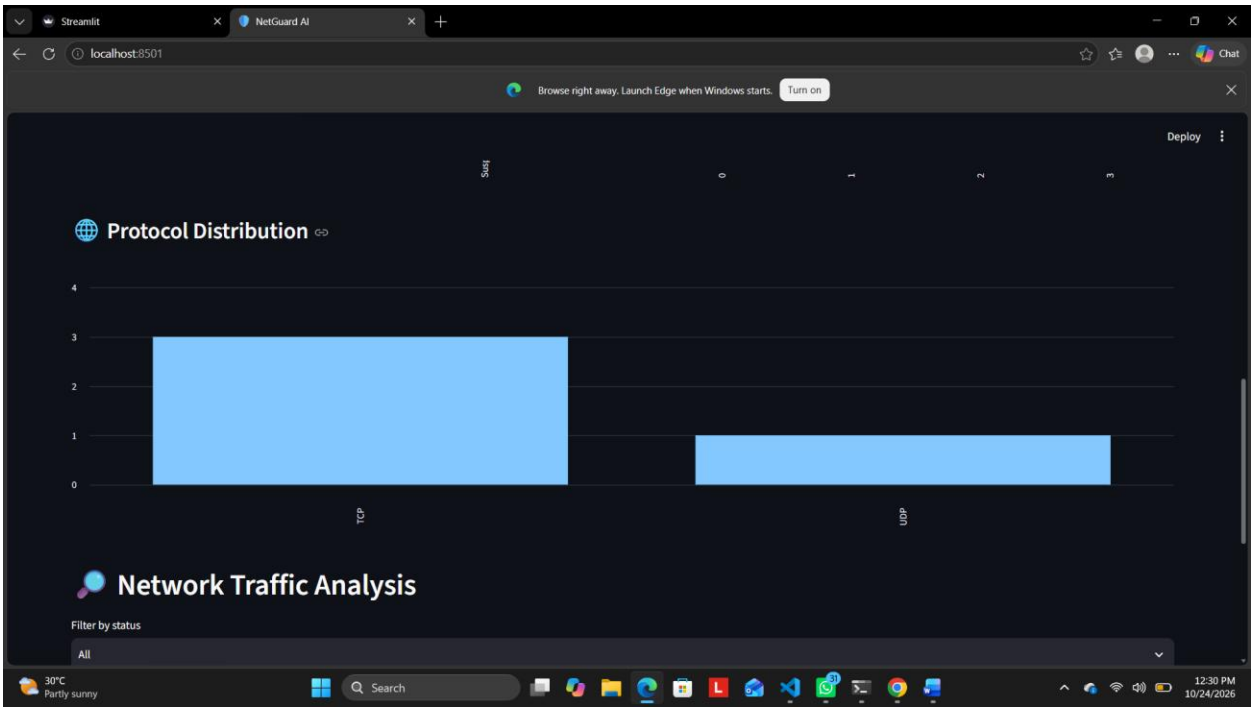
The final report should include the actual counts and calculated values obtained from the completed evaluation. No performance percentage should be reported without calculating it from the test results. If a denominator is zero, the corresponding metric should be reported as undefined.

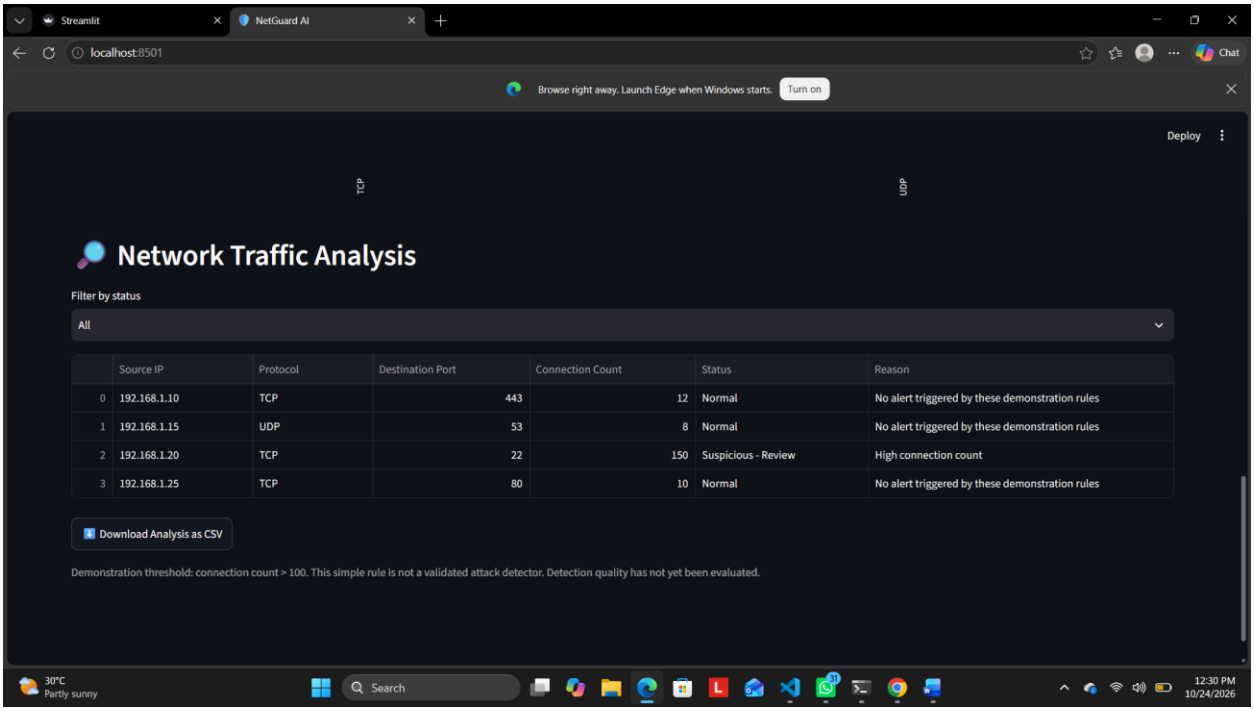
Because the dataset is small and simulated, any measured results represent an initial prototype evaluation rather than evidence of real-world effectiveness. A larger and independently labelled dataset would be needed for a more reliable assessment.

10. Result and Analysis






Streamlit
NetGuard AI
localhost:8501
Chat
Browse right away. Launch Edge when Windows starts.
Turn on
Deploy
Protocol Distribution
4
3
2
1
0
TCP
UDP
Network Traffic Analysis
Filter by status
All
30°C
Partly sunny
Search
12:30 PM
10/24/2026


Streamlit
NetGuard AI
localhost:8501
Chat
Browse right away. Launch Edge when Windows starts.
Turn on
Deploy
TCP
UDP
Network Traffic Analysis
Filter by status
All
Source IP
Protocol
Destination Port
Connection Count
Status
Reason
0 192.168.1.10 TCP 443 12 Normal No alert triggered by these demonstration rules
1 192.168.1.15 UDP 53 8 Normal No alert triggered by these demonstration rules
2 192.168.1.20 TCP 22 150 Suspicious - Review High connection count
3 192.168.1.25 TCP 80 10 Normal No alert triggered by these demonstration rules
Download Analysis as CSV
Demonstration threshold: connection count > 100. This simple rule is not a validated attack detector. Detection quality has not yet been evaluated.
30°C
Partly sunny
Search
12:30 PM
10/24/2026

11. Network Security Considerations and Limitations

Detection latency: The prototype processes uploaded CSV files and does not provide continuous live packet monitoring or guaranteed real-time alerts.

False positives: Legitimate traffic may match a configured threshold or selected-port rule. Alerts therefore require human review.

False negatives: Activity that does not match the configured rules may go undetected. Low-rate or distributed flooding and attacks requiring authentication or protocol context are not reliably identified by the current features.

Data quality: Missing, invalid or inconsistent fields can affect the analysis. Classification is limited by the information present in the input file.

Scalability: Testing with a small sample does not establish how the application will perform on very large datasets or high-throughput networks.

Privacy: Traffic logs may contain IP addresses and operational metadata. Access should be restricted, sensitive data should be handled carefully, and private traffic captures should not be published in a public repository.

Security interpretation: A suspicious status is an indicator for investigation, not proof of compromise. The prototype should not be treated as a replacement for professional monitoring, endpoint protection or a fully validated intrusion detection system.

12. Future Enhancements

Several improvements could increase the prototype's usefulness.

First, the application could include additional traffic features such as flow duration, packet counts, packet rate and time-based connection patterns, provided these fields are available from a suitable data source.

Second, future versions could support authorized live traffic ingestion and more detailed protocol analysis. This would require careful attention to privacy, performance and access controls.

Third, the evaluation could be expanded using larger datasets with reliable labels. This would allow more meaningful comparisons of detection rates, false positives and missed cases.

Fourth, a machine-learning-based anomaly detector could be integrated after resolving the environment and dependency issues. Such a model would need suitable training or reference data, testing and honest evaluation before its results could be relied upon.

Finally, configurable thresholds, richer incident reports and improved alert explanations could make the application more flexible and easier to use.

13. Conclusion

NetGuardAI demonstrates a practical approach to preliminary network traffic analysis using Python, pandas and Streamlit. It accepts structured CSV records, applies rule-based checks, classifies potentially suspicious activity and provides explanations through an interactive dashboard.

The prototype supports the examination of normal traffic, scan-like patterns, high connection counts, selected destination ports and incomplete or ambiguous records. Its visual summaries, filters and downloadable results make the analysis easier to review.

The project also illustrates important limitations of simple intrusion detection rules. Thresholds can generate false positives, attacks may evade the configured conditions, and the available data may not contain enough information to establish malicious intent. The current version does not provide live monitoring or a functioning machine-learning detector.

Overall, NetGuardAI serves as an educational cybersecurity prototype and a foundation for future development. More comprehensive datasets, richer traffic features, stronger evaluation and carefully validated detection techniques would be needed before using the system for operational security monitoring.

14. References

1. Python Software Foundation. Python Documentation. https://docs.python.org/3/

2. Streamlit. Streamlit Documentation. https://docs.streamlit.io/

3. pandas. pandas Documentation. https://pandas.pydata.org/docs/

4. Github: Github helped me to organize project files. I used Github resources for learning.

5. Use of AI Assistance: ChatGPT was used as an AI-assistance. It helped me understand technical concepts and prepare explanations and the python code.